



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U02_A03 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 9 de Octubre, 2025

ÍNDICE

INTRODUCCIÓN3

ENUNCIADO.....4

 Probad todo el código del Apartado 34

 Probad los Scripts del Apartado.....8

 Caso práctico 3.....16

 Objetivos y prototipos.....16

 Ponte a prueba 3.....16

 This16

 Clave de Resolución:17

DESARROLLO DE LA PRACTICA.....18

 Descripción18

 Contenido del archivo18

 • Código del Apartado 3 de teoría18

 • Scripts adjuntos21

 • Caso práctico 3 – Objetos y prototipos.....21

 • Ponte a prueba – this.....22

 Cómo visualizarlo.....23

 Requisitos:23

 Pasos:.....23

 Objetivo de la práctica.....23

 • Probar el código del material teórico del Apartado 3.23

 • Ejecutar y analizar los scripts proporcionados en la tarea.....23

 • Crear objetos usando clases (en el caso de mi práctica), funciones constructoras y
Object.create().....23

 • Analizar el comportamiento de this en distintos contextos.....23

 • Documentar correctamente cada ejercicio con nombre, fecha y capturas.....23

 • Aplicar buenas prácticas de programación y presentación técnica.....23

 • Entregar el proyecto en formato PDF junto con los archivos fuente y pantallazos.23

CONCLUSIÓN24

INTRODUCCIÓN

En el “Probad todo el código del Apartado 3” deberemos incluir en uno o varios scripts absolutamente todo el código que aparezca en la teoría del apartado 3, así como su ejecución y prueba de autoría.

“En el Probad los Scripts del Apartado 3” ejecutaremos todo el código presente en los scripts adjuntos y proporcionados en la tarea y explicaremos su funcionamiento.

En el “Caso Práctico 3” deberemos crear la siguiente estructura (imagen en el ejercicio en Aules) y para ello crearemos el código que sea necesario.

En el “Ponte a Prueba 3” analizaremos el código adjunto y deberemos indicar qué resultado se mostrará.

ENUNCIADO

Probad todo el código del Apartado 3

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Captura de pantalla – DWECS_U02_A03_01.html

```
<> DWECS_U02_A03_01.html x
<> DWECS_U02_A03_01.html > html > body
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 07/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>3.1. Objetos: principios básicos</title>
10   </head>
11   <body>
12     <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14     <script>
15       // 3.1. Objetos: principios básicos
16       let coche = {
17         color: "rojo",
18         marca: "seat",
19         arrancar: function() {
20           console.log("Arrancando");
21         }
22       }
23
24       console.log(coche.color); // rojo
25       console.log(coche.marca); // seat
26       coche.arrancar(); // Arrancando
27
28       coche.modelo = "ibiza";
29       coche["combustible"] = "diesel";
30
31       for(let prop in coche) {
32         console.log(prop);
33       }
34
35       // color
36       // marca
37       // arrancar
38
39       let persona = {
40         edad: 20,
41         nombre: "David"
42       }
43
44       console.log("edad" in persona); // true
45       console.log("apellido" in persona); // false
46     </script>
47
48     <!-- Prueba de Autoría -->
49     <p>Autor: Mario Segura Abad<br></p>
50     <p>Fecha de realización: 07/10/2025</p>
51   </body>
52 </html>
```

Captura de pantalla – DWEC_U02_A03_02.html

DWEC_U02_A03_01.htmlDWEC_U02_A03_02.html X

DWEC_U02_A03_02.html > html > body > p

```
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 07/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>3.4. Objetos predefinidos y métodos de primitivas</title>
10   </head>
11   <body>
12     <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14     <script>
15       // 3.4. Objetos predefinidos y métodos de primitivas
16       let cadena = "Texto";
17       console.log(cadena.length); // 5
18
19       Number(2.54).toExponential(); // '2.54e+0'
20       "texto".toUpperCase(); // "TEXT0"
21       Number(25).toString(); // "25"
22
23       let cadena2 = "Esto es un texto";
24       cadena2.indexOf("x"); // 13
25       cadena2.includes("texto"); // true
26       cadena2.startsWith("Es"); // true
27       cadena2.endsWith("Es"); // false
28     </script>
29
30     <!-- Prueba de Autoría -->
31     <p>Autor: Mario Segura Abad<br></p>
32     <p>Fecha de realización: 07/10/2025</p>
33   </body>
34 </html>
```

Capturas de pantalla – DWEC_U02_A03_03.html

DWEC_U02_A03_02.htmlDWEC_U02_A03_03.html XDWEC_U02_A03_04.htmlDWEC_U02_A03_05.html

DWEC_U02_A03_03.html > ...

```
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 07/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>3.6. This</title>
10   </head>
11   <body>
12     <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14     <script>
15       // 3.6. This
16       let usuario = {
17         nombre: "David",
18         saluda: function() { // Lugar de definición de "saluda"
19           return this.nombre;
20         }
21       }
22
23       // Lugar de ejecución "saluda"
24       // Se llama llama a la función sobre el objeto
25       usuario.saluda(); // David
26
27       // Lugar de ejecución "saluda"
28       // Se llama a la función sin referencia al objeto
29       let intentaSaludar = usuario.saluda();
30       intentaSaludar(); // undefined
31
32       let usuario2 = {
33         nombre: "David",
34         saluda: () => { // Función 'saluda' definida como función flecha
35           return this.nombre;
36         }
37       }
38
39       // Lugar de ejecución de "saluda"
40       // Se llama a la función sobre el objeto
41       // 'this' no está definido, por ser una función flecha
42       usuario2.saluda(); // undefined
43
44       let usuario3 = {
45         nombre: "David",
46         saludaConRetardo: function() {
47           setTimeout(function() {
48             // Intenta mostrar el nombre del usuario
49             console.log(this.nombre);
50           }, 1000); // Retardo de 1000ms (1s)
51         }
52       }
53     </script>
```

```
90
91     <!-- Prueba de Autoría -->
92     <p>Autor: Mario Segura Abad<br></p>
93     <p>Fecha de realización: 07/10/2025</p>
94 </body>
95 </html>
```

Captura de pantalla – DWEC_U02_A03_04.html

```
<> DWEC_U02_A03_03.html <> DWEC_U02_A03_04.html X
<> DWEC_U02_A03_04.html > html > body > p
1 <!-- Autor: Mario Segura Abad -->
2 <!-- Fecha: 07/10/2025-->
3
4 <!DOCTYPE html>
5 <html lang="es">
6   <head>
7     <meta charset="UTF-8">
8     <meta name="viewport" content="width=device-width, initial-scale=1.0">
9     <title>3.7. Prototipos y herencia</title>
10  </head>
11  <body>
12    <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14    <script>
15      // 3.7. Prototipos y herencia
16      let usuario = {
17        nombre: "David"
18      }
19
20      // El método 'toLocaleString' funciona, pese a no haberlo definido
21      // Esta función está definida en 'Object.prototype'
22      usuario.toLocaleString(); // '[object Object]'
23    </script>
24
25    <!-- Prueba de Autoría -->
26    <p>Autor: Mario Segura Abad<br></p>
27    <p>Fecha de realización: 07/10/2025</p>
28  </body>
29 </html>
```

Captura de pantalla – DWECE_U02_A03_05.html

```
<> DWECE_U02_A03_04.html <> DWECE_U02_A03_05.html x
<> DWECE_U02_A03_05.html > html > body > script > Usuario > saludar
1 <!-- Autor: Mario Segura Abad -->
2 <!-- Fecha: 07/10/2025-->
3
4 <!DOCTYPE html>
5 <html lang="es">
6   <head>
7     <meta charset="UTF-8">
8     <meta name="viewport" content="width=device-width, initial-scale=1.0">
9     <title>3.9. Clases</title>
10  </head>
11  <body>
12    <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14    <script>
15      // 3.9. Clases
16
17      // Definición de clase
18      class Usuario {
19        constructor(nombre, apellidos) {
20          this.nombre = nombre;
21          this.apellidos = apellidos;
22        }
23
24        // No hay coma entre las definiciones de los métodos
25
26        saludar() {
27          console.log(`Me llamo ${this.nombre} ${this.apellidos}`);
28        }
29      }
30
31      // Creación de un objeto basado en la clase
32      // (En realidad, es un objeto cuyo prototipo es el objeto Usuario)
33      let usuario = new Usuario("David", "Pérez");
34      usuario.saludar(); // Me llamo David Pérez
35    </script>
36
37    <!-- Prueba de Autoría -->
38    <p>Autor: Mario Segura Abad<br></p>
39    <p>Fecha de realización: 07/10/2025</p>
40  </body>
41 </html>
```

Captura de pantalla – DWECE_U02_A03_06.html

```
<> DWECE_U02_A03_05.html <> DWECE_U02_A03_06.html x
<> DWECE_U02_A03_06.html > html > body > script
1 <!-- Autor: Mario Segura Abad -->
2 <!-- Fecha: 07/10/2025-->
3 The meta element represents various kinds of metadata that cannot be expressed using the
4 <!DOCTYPE <> Widely available across major browsers (Baseline since 2015)
5 <html lan
6   <head MDN Reference
7     <meta charset="UTF-8">
8     <meta name="viewport" content="width=device-width, initial-scale=1.0">
9     <title>3.10. JSON</title>
10  </head>
11  <body>
12    <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14    <script>
15      // 3.10. JSON
16      let usuario = {
17        nombre: "David",
18        edad: 25,
19        permisos: ["administrador", "editor"]
20      }
21
22      let cadenaJson = JSON.stringify(usuario);
23      // cadenaJson es un string
24
25      let usuarioRecuperado = JSON.parse(cadenaJson);
26      // usuarioRecuperado es un objeto
27    </script>
28
29    <!-- Prueba de Autoría -->
30    <p>Autor: Mario Segura Abad<br></p>
31    <p>Fecha de realización: 07/10/2025</p>
32  </body>
33 </html>
```

Probad los Scripts del Apartado

Ejecutad los scripts adjuntos y explicad su funcionamiento. Recordad incluir pruebas de autoría.

Salida por consola – DWEK_U02_A03_01.js

```

> // Autor: Mario Segura Abad
// Fecha: 07/10/2025

// Creamos un array llamado array1 con tres valores tipo string
let array1 = ["valorA", "valorB", "valorC"];

// -----
// DESESTRUCTURACIÓN DE ARRAYS
// -----

// En los arrays, los valores se extraen según su posición (índice 0, 1, 2...)
// Aquí se están creando tres variables: a, b y c
// Cada una tomará el valor correspondiente del array en orden.
let [a, b, c] = array1;

// Mostramos en consola el contenido de las variables:
console.log(a); // valorA -> primer elemento del array
console.log(b); // valorB -> segundo elemento del array
console.log(c); // valorC -> tercer elemento del array

// -----
// IGNORAR ELEMENTOS DEL ARRAY
// -----

// Podemos "saltar" posiciones usando comas sin asignar variable.
// En este caso:
// d toma el primer valor del array ("valorA")
// se ignora el segundo valor (por la coma sin variable)
// e toma el tercer valor ("valorC")
let [d, , e] = array1;

console.log(d); // valorA -> primer elemento
console.log(e); // valorC -> tercer elemento (hemos ignorado el segundo)

// -----
// OPERADOR REST (...)
// -----

// El operador rest (...) agrupa los elementos restantes en un nuevo array.
// f toma el primer valor ("valorA")
// g toma el resto de los elementos ["valorB", "valorC"]
let [f, ...g] = array1;

console.log(f); // valorA -> primer elemento
console.log(g); // ["valorB", "valorC"] -> el resto del array empaquetado

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 07/10/2025");
valorA
valorB
valorC
valorA
valorC
valorA
  ▶ (2) ['valorB', 'valorC']
Autor: Mario Segura Abad
Fecha de realización: 07/10/2025

```


Salida por consola – DWEK_U02_A03_02.js

```
top Filter
> // Autor: Mario Segura Abad
// Fecha: 07/10/2025

// Creamos un objeto llamado 'objeto1' con tres propiedades (clave: valor)
let objeto1 = {
  pA: "valorA",
  pB: "valorB",
  pC: "valorC"
};

// -----
// DESESTRUCTURACIÓN DE OBJETOS
// -----

// En los objetos, las variables se asocian por **nombre de propiedad**, no por posición.
// Es decir, las variables pA, pB y pC tomarán los valores de las propiedades con el mismo nombre.
let {pA, pB, pC} = objeto1;

// Mostramos los valores extraídos:
console.log(pA); // valorA -> valor de la propiedad "pA"
console.log(pB); // valorB -> valor de la propiedad "pB"
console.log(pC); // valorC -> valor de la propiedad "pC"

// -----
// OPERADOR REST (...) CON OBJETOS
// -----

// Declaramos una variable 'resto' que usaremos más abajo
let resto;
```

```
top Filter

// IMPORTANTE: En este punto, la variable 'pA' ya ha sido declarada anteriormente.
// Si intentáramos usar 'let {pA, ...resto} = objeto1;' otra vez, daría error
// porque estaríamos redeclarando 'pA' en el mismo ámbito.

// Para evitar ese error, usamos una **asignación entre paréntesis**, sin 'let'.
// Esto indica que no estamos declarando nuevas variables, sino reasignando valores existentes.
({pA, ...resto} = objeto1); // Utilizar el operador rest: 'resto' almacenará un objeto

// En esta línea:
// - 'pA' toma el valor "valorA"
// - 'resto' recibe un **nuevo objeto** con las propiedades restantes del original
//   (en este caso, pB y pC)
console.log(pA); // valorA
console.log(resto); // {pB: "valorB", pC: "valorC"}

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha: 07/10/2025");
valorA
valorB
valorC
valorA
  ▶ {pB: 'valorB', pC: 'valorC'}
Autor: Mario Segura Abad
Fecha: 07/10/2025
< undefined
```

Salida por consola – DWEK_U02_A03_03.js

top

Filter

```
> // Autor: Mario Segura Abad
// Fecha: 07/10/2025

// Creamos un objeto con una propiedad 'nombre'
let objeto1 = {nombre: "David"}; // Se crea un objeto
// En este momento, 'objeto1' guarda una REFERENCIA al objeto {nombre: "David"} en memoria

// Asignamos 'objeto1' a 'objeto2'
let objeto2 = objeto1;
// Aquí NO se crea un nuevo objeto
// Ambos ('objeto1' y 'objeto2') apuntan al MISMO objeto en memoria

// Creamos dos nuevos objetos vacíos (cada uno ocupa su propio espacio en memoria)
let objeto3 = {}; // objeto3 apunta a un nuevo objeto vacío
let objeto4 = {}; // objeto4 apunta a otro nuevo objeto vacío

// Copiamos las propiedades de 'objeto1' dentro de 'objeto3'
// Object.assign(destino, origen) -> copia los pares clave: del origen al destino
Object.assign(objeto3, objeto1);

// Copiamos las propiedades de 'objeto1' dentro de 'objeto4' y además añadimos 'apellido: "Pérez"'
Object.assign(objeto4, objeto1, {apellido: "Pérez"}); // Copia de 'objeto1' a 'objeto4' junto con una propiedad adicional
// Resultado: objeto4 = {nombre: "David", apellido: "Pérez"}

// Cambiamos el valor de la propiedad 'nombre' en 'objeto1'
objeto1.nombre = "Juan";
// Este cambio también afecta a 'objeto2', porque ambos apuntan al mismo objeto

// Cambiamos el valor de 'nombre' solo en 'objeto4'
objeto4.nombre = "Pablo";
// Este cambio NO afecta a 'objeto1' ni 'objeto2', porque 'objeto4' es un objetivo distinto

// Mostramos resultados:
console.log( objeto2.nombre ); // Juan ('objeto2' apunta al mismo objeto que 'objeto1')
console.log( objeto3.nombre ); // David ('objeto3' apunta a un objeto distinto)

// Ahora anulamos las referencias
objeto1 = null; // 'objeto1' ya no apunta al objeto. El objeto sigue existiendo.
objeto2 = null; // 'objeto2' ya no apunta al objeto. El primer objeto se elimina porque ya no está referenciado

// En este momento:
// El objto original {nombre: "Juan"} ya no tiene ninguna variable que lo referencie
// Por tanto, el recolector de basura (garabage collector) lo eliminará automáticamente
// porque ya no es accesible desde el código

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 07/10/2025");
```

Juan

David

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

< undefined

Salida por consola – DWEK_U02_A03_04.js

top

Filter

```
> // Autor: Mario Segura Abad
// Fecha: 07/10/2025

// Creamos un array inicial con tres elementos
let array1 = ['a', 'b', 'c'];

// -----
// MÉTODOS push() y pop()
// -----

// push() -> Añade un elemento al final del array
array1.push("d"); // array1 = [ 'a', 'b', 'c', 'd' ]

// pop() -> Elimina el último elemento del array
array1.pop(); // array1 = [ 'a', 'b', 'c' ]
// Devuelve el elemento eliminado ("d")

// -----
// MÉTODOS shift() y unshift()
// -----

// shift() -> Elimina el PRIMER elemento del array
// Devuelve el elemento eliminado ("a")
array1.shift(); // array1 = [ 'b', 'c' ] (la función devuelve 'a')

// unshift() -> Añade uno o varios elementos al INICIO del array
array1.unshift("a"); // array1 = [ 'a', 'b', 'c' ]
```

top Filter

```
// -----
// MÉTODO splice()
// -----

// Creamos un nuevo array
let array2 = [10, 11, 12, 13, 14];

// splice(posicionInicial, númeroDeElementosAEliminar)
// Elimina 2 elementos a partir de la posición 1
array2.splice(1, 2); // array2 = [ 10, 13, 14 ] (extrae 2 elementos a partir de la posición 1)
// Elimina los elementos [11, 12]
// array2 queda como [10, 13, 14]
// splice() modifica el array original

// -----
// MÉTODO slice()
// -----

// Creamos otro array
let array3 = [1, 2, 3, 4, 5];

// slice(inicio, fin) -> Extrae una "copia" de una parte del array
// No modifica el array original
let array4 = array3.slice(1, 4);
// Extrae los elementos desde el índice 1 hasta el 4 (sin incluir el 4)
// array4 = [2, 3, 4]
// array3 sigue siendo [1, 2, 3, 4, 5]
```

Salida por consola – DWEC_U02_A03_05.js

top Filter

```
> // Autor: Mario Segura Abad
// Fecha: 07/10/2025

// -----
// FUNCIÓN CON 'this' Y PARÁMETROS
// -----

// Esta función recibe dos parámetros y usa 'this' para acceder a una propiedad
function saludar(mensajeInicio, mensajeFinal) {
    return mensajeInicio + this.nombre + mensajeFinal;
    // 'this' hace referencia al objeto que esté "vinculado" en el momento de la llamada
}

// -----
// CREAMOS UN OBJETO PARA HACER PRUEBAS
// -----
let usuario1 = {nombre: "David"};

// -----
// 1. LLAMADA NORMAL SIN VINCULAR 'this'
// -----
saludar("Hola ", " ¿qué tal estás?"); // Hola undefined, ¿qué tal estás?
// Explicación:
// - Como no se ha pasado ningún objeto, 'this' no apunta a 'usuario1'.
// - En modo estricto, 'this' sería undefined.
// - Por tanto, 'this.nombre' no existe -> devuelve undefined.

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 07/10/2025");

// Ahora, aunque llamemos a la nueva función sin pasar 'this' explícitamente,
// éste ya quedó vinculado de forma permanente a 'usuario1'
saludaADavid("Hola ", " ¿qué tal estás?"); // Hola David, ¿qué tal estás?

Autor: Mario Segura Abad
Fecha de realización: 07/10/2025
< 'Hola David, ¿qué tal estás?'
```

Salida por consola – DWEK_U02_A03_06.js

🔍 | 🔍 | top ▼ | 🔍 Filter

```
> // Autor: Mario Segura Abad
// Fecha: 07/10/2025

// -----
// FUNCIÓN CONSTRUCTORA
// -----

// En JavaScript, una función constructora sirve para crear múltiples objetos
// del mismo "tipo". Por convención, se escribe con la primera letra en mayúscula.
function Usuario(nombre, apellidos) {

    // Las propiedades definidas con 'this' se asignan al nuevo objeto que se cree
    this.nombre = nombre;
    this.apellidos = apellidos;

    // Se define también un método (función) como propiedad del objeto
    this.saludar = function() {
        console.log(`Hola, soy ${this.nombre} ${this.apellidos}`);
    }

    // 'nombre', 'apellidos' y 'saludar' se convierten en propiedades propias
    // de cada objeto creado con 'newUsuario(...)'
}
```

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 07/10/2025");
```

Hola, soy David Pérez

Hola, soy Jorge López

David Pérez se despide

Jorge López se despide

Desconocida

Desconocida

Calle Nueva

Desconocida

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

⏪ undefined

Salida por consola – DWEK_U02_A03_07.js

top ▾

Filter

```
> // Autor: Mario Segura Abad
// Fecha: 08/10/2025

// -----
// 1. OBJETO BASE (PROTOTIPO)
// -----

// Se crea un objeto normal que servirá como prototipo de otros objetos
let Usuario = {
  nombre: "",
  apellido: ""
}

// -----
// 2. MÉTODOS DEFINIDOS EN EL PROTOTIPO
// -----

// Método para inicializar los datos del usuario
Usuario.setDatos = function(nombre, apellidos) {
  this.nombre = nombre; // 'this' hace referencia al objeto que lo llama
  this.apellidos = apellidos;
}

// Método para saludar
Usuario.saludar = function() {
  console.log(`Hola, soy ${this.nombre} ${this.apellidos}`);
}

// Esto es el "encadenamiento del prototipo" o "prototype chain".

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/10/2025");

Hola, soy David Pérez
Hola, soy Jorge López
David Pérez se despide
Jorge López se despide
Desconocida
Desconocida
Calle Nueva
Desconocida
Autor: Mario Segura Abad
Fecha de realización: 08/10/2025
```

Salida por consola – DWEK_U02_A03_08.js

🔍 | top ▼ | 👁 | 🔍 Filter

```
> // Autor: Mario Segura Abad
// Fecha: 08/10/2025

// Se define una clase base llamada 'UsuarioBase'
class UsuarioBase {
  // El constructor se ejecuta cuando se crea un nuevo objeto con 'new UsuarioBase(...)'
  constructor(nombre, apellidos) {
    // Se inicializan las propiedades 'nombre' y 'apellidos' del objeto
    this.nombre = nombre;
    this.apellidos = apellidos;
  }

  // Método 'saludar' definido dentro de la clase
  // No pertenece a cada objeto directamente, sino al prototipo de la clase 'UsuarioBase'
  saludar() {
    console.log(`Me llamo ${this.nombre} ${this.apellidos}`);
  }
}

// Se crea una nueva clase llamada 'UsuarioEspecial' que hereda de 'UsuarioBase'
class UsuarioEspecial extends UsuarioBase {

  // Método 'cantar' definido dentro de 'UsuarioEspecial'
  cantar() {
    console.log("Yo también sé cantar");
  }
}

// JavaScript busca el método 'saludar' en el objeto 'usuario':
// 1. Primero busca en el propio objeto ('usuario') y no lo encuentra.
// 2. Luego busca en su prototipo, que es 'UsuarioEspecial.prototype', y tampoco lo encuentra.
// 3. Finalmente busca en el prototipo 'UsuarioEspecial', que es 'UsuarioBase.prototype', donde sí lo encuentra.
// Por tanto, se ejecutará el método 'saludar' de la clase 'UsuarioBase'
usuario.saludar(); // Me llamo David Pérez

// JavaScript busca el método 'cantar' en el objeto 'usuario':
// 1. No está en el propio objeto ('usuario').
// 2. A continuación lo busca en su prototipo, 'UsuarioEspecial.prototype', y allí lo encuentra.
// Por tanto, se ejecuta el método 'cantar' definido en 'UsuarioEspecial'.
usuario.cantar(); // Yo también sé cantar

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/10/2025");
```

Me llamo David Pérez

Yo también sé cantar

Autor: Mario Segura Abad

Fecha de realización: 08/10/2025

Salida por consola – DWEK_U02_A03_09.js

```

> // Autor: Mario Segura Abad
  // Fecha: 08/10/2025

// Se define una clase base llamada 'Usuario'
class Usuario {
  // El constructor se ejecuta cuando se crea un objeto nuevo con 'newUsuario(...)'
  constructor(nombre, apellidos) {
    // Se inicializan las propiedades 'nombre' y 'apellidos' del nuevo objeto
    this.nombre = nombre;
    this.apellidos = apellidos;
  }

  // Método 'saludar' definido en 'Usuario.prototype'
  // No pertenece a cada objeto directamente, sino al prototipo
  saludar() {
    console.log(`Me llamo ${this.nombre} ${this.apellidos}`);
  }
}

// Se define una nueva clase 'UsuarioEspecial' que hereda de 'Usuario'
class UsuarioEspecial extends Usuario {

  // Se define un nuevo constructor que amplía el de la clase padre
  // Añadiendo un tercer parámetro: 'edad'
  constructor(nombre, apellidos, edad) {

    // Se llama al constructor de la clase padre ('Usuario') mediante 'super'
    // Es OBLIGATORIO hacerlo antes de usar 'this' en una clase que hereda
    // Esto inicializa las propiedades 'nombre' y 'apellidos' en el nuevo objeto
    super(nombre, apellidos);

    // Al ejecutar 'usuario.saludar()', JavaScript hace lo siguiente:
    //
    // 1. Busca el método 'saludar' en el propio objeto ('usuario') -> no existe ahí.
    // 2. Lo busca en su prototipo ('UsuarioEspecial.prototype') -> lo encuentra.
    // 3. Dentro del método, se ejecuta primero 'super.saludar()', que llama
    //    al método 'saludar' de 'Usuario.prototype'.
    // 4. Luego ejecuta la segunda línea: console.log(`Y tengo ${this.edad} años`);
    //
    // Resultado final en consola:
    usuario.saludar(); // Me llamo David Pérez
    // Y tengo 35 años

    // Prueba de Autoría
    console.log("Autor: Mario Segura Abad");
    console.log("Fecha de realización: 08/10/2025");
  }
}

Me llamo David Pérez

Y tengo 35 años

Autor: Mario Segura Abad

Fecha de realización: 08/10/2025
```

Caso práctico 3

Objetivos y prototipos

Crea el código necesario para crear la siguiente estructura de prototipos:

Se deben poder crear objetos basados en cualquiera de los tres prototipos indicados. Puedes utilizar cualquiera de los tres sistemas estudiados:

- Sintaxis **class**.
- Funciones constructoras.
- Objetos enlazados con **Object.create**.

Ponte a prueba 3

This

Analiza el siguiente código e indica qué resultado se mostrará en las seis líneas indicadas.

```
function Persona(nombre, apellido) {  
    this.nombre = nombre;  
    this.apellido = apellido;  
    this.saludar = function () {  
        return `${this.nombre} ${this.apellido}`;  
    }  
}
```

```
let persona1 = new Persona("Ana", "Pérez");  
let persona2 = new Persona("Lucía", "Martínez");  
let persona3 = new Persona("Inés", "González");
```

```
let saludo1 = persona1.saludar.bind(persona3);  
let saludo2 = persona2.saludar;
```

```
persona1.saludar() ;           // 1  
persona2.saludar() ;           // 2  
persona1.saludar.call(persona3); // 3  
persona1.saludar.apply(persona2); // 4  
saludo1();                     // 5  
saludo2();                     // 6
```


Clave de Resolución:

saludar hace referencia a this. Recuerda que this es una referencia dinámica que no depende del lugar de declaración de la función, sino de dónde y cómo se produzca la llamada dicha función.

DESARROLLO DE LA PRACTICA

Práctica DVEC – Unidad 2

Descripción

Este proyecto recoge los ejercicios prácticos del Apartado 3 de la teoría de JavaScript, centrados en la creación de objetos y el uso de prototipos. Se han implementado tres formas distintas de crear objetos: mediante **clases**, **funciones constructoras** y **Object.create()**. Además, se ha analizado el comportamiento de **this** en distintos contextos, entendiendo cómo varía según el lugar y forma de invocación.

Todo el código teórico ha sido probado, documentado y acompañado de capturas de ejecución. El proyecto se entrega en formato PDF junto con los archivos fuente y pantallazos, comprimidos en un archivo ZIP.

Contenido del archivo

- **Código del Apartado 3 de teoría**
Se ha recopilado y probado todo el código mostrado en clase, incluyendo ejemplos de creación de objetos, uso de prototipos, herencia y métodos. Cada archivo incluye comentarios con el nombre del autor y fecha de realización. Se han capturado los resultados por consola como prueba de ejecución.
Ejemplo de ejecución:
Salida por consola – DVEC_U02_A03_01.html

Abrir la consola (F12) para ver el resultado

e.g. /eventd/ -cdn url:a.com

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

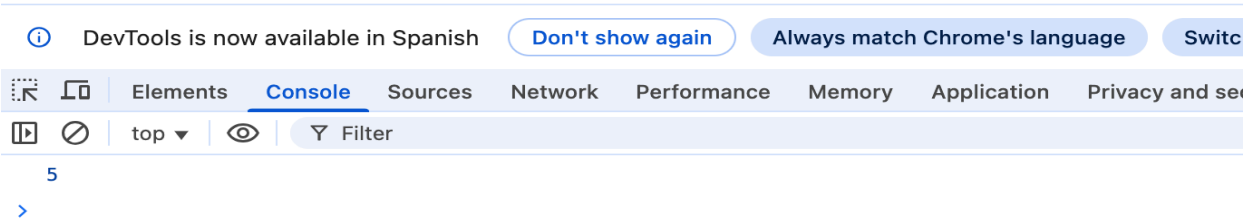


Salida por consola – DWEC_U02_A03_02.html

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

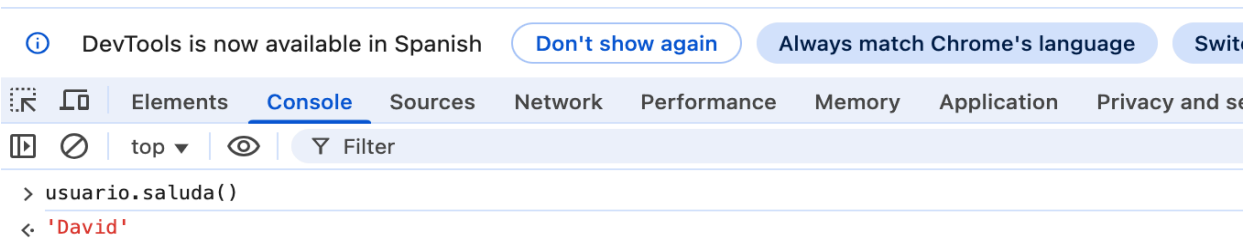


Salida por consola – DWEC_U02_A03_03.html

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

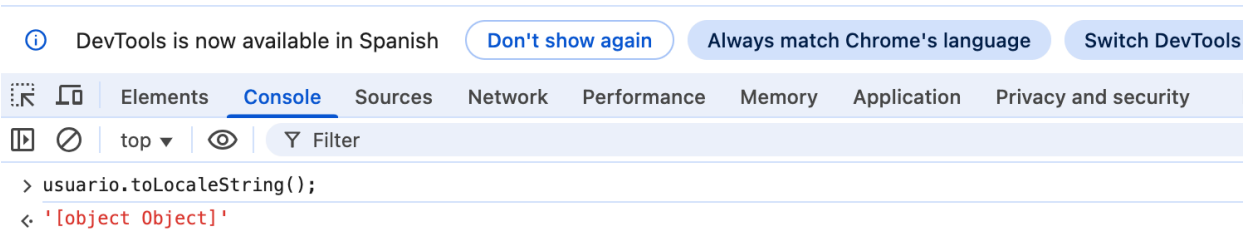


Salida por consola – DWEC_U02_A03_04.html

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

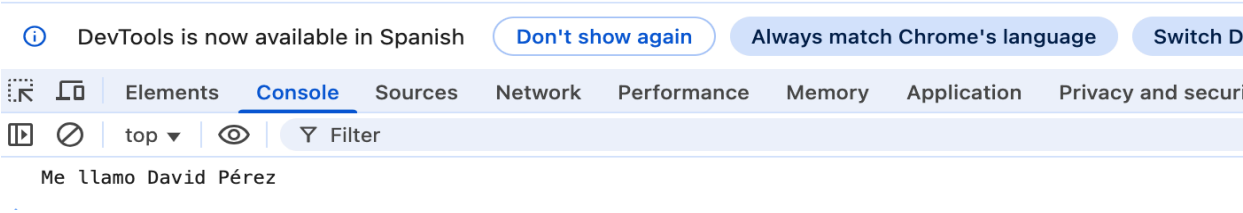


Salida por consola – DWEC_U02_A03_05.html

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025

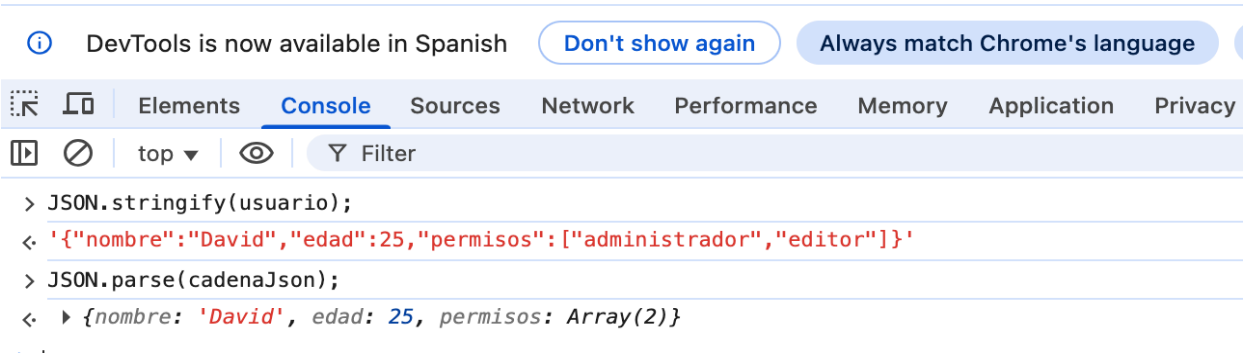


Salida por consola – DWEC_U02_A03_06.html

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 07/10/2025



- **Scripts adjuntos**

Se han ejecutado los scripts proporcionados en la propia tarea, explicando su funcionamiento línea por línea.

Se han documentado los resultados obtenidos y se han obtenido pantallazos con el nombre del autor y la fecha.

- **Caso práctico 3 – Objetos y prototipos**

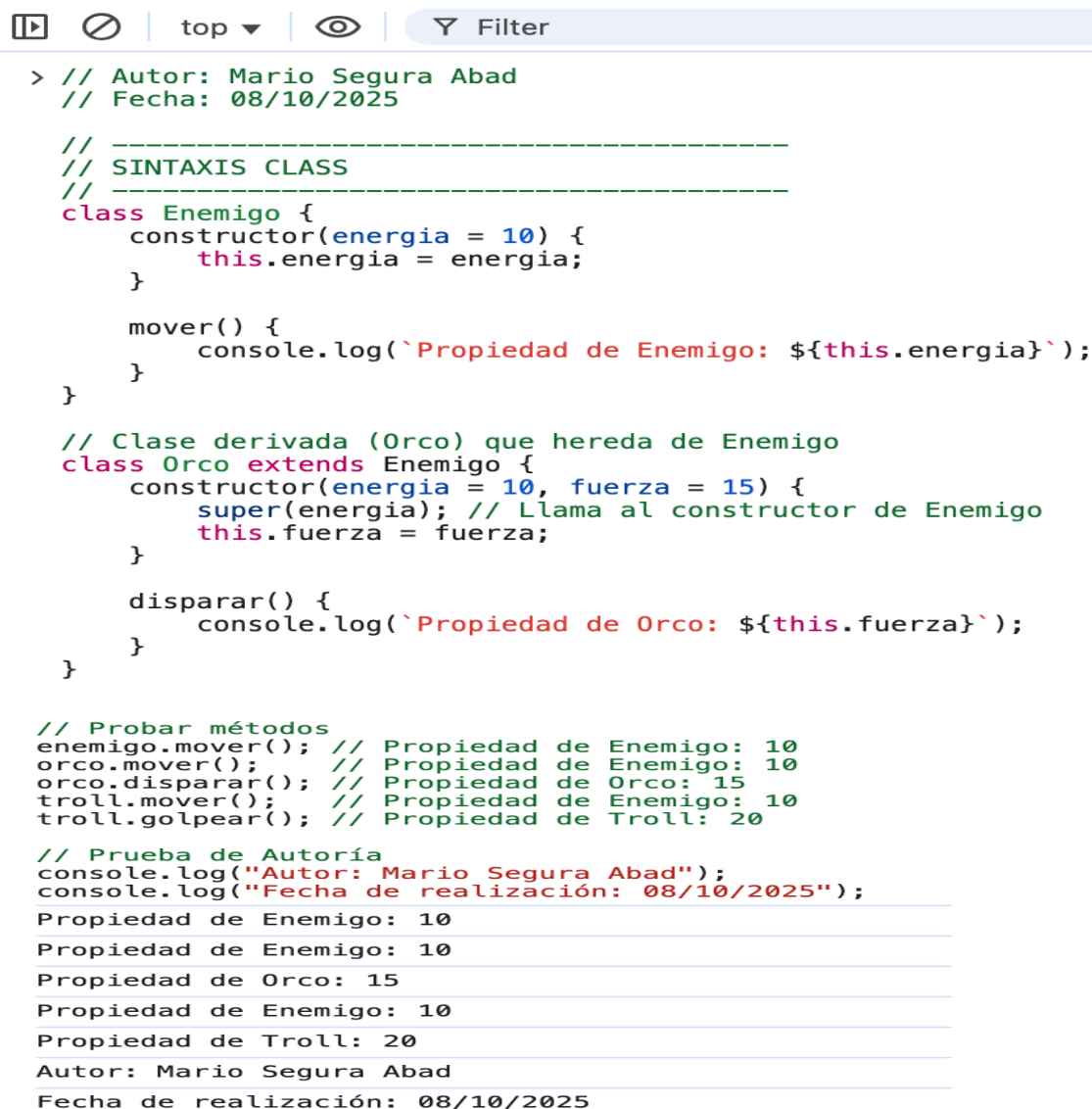
En este ejercicio se ha trabajado uno de los pilares fundamentales de JavaScript: la creación de objetos y el uso de prototipos para establecer relaciones de herencia y reutilización de métodos. El objetivo ha sido construir una estructura que permita crear instancias de objetos basados en tres enfoques distintos, en el caso de mi práctica syntax class, todos válidos y funcionales dentro del lenguaje:

1. Sintaxis class

La sintaxis class es la más moderna y legible. Aunque internamente sigue utilizando prototipos, permite definir clases de forma más clara.

Este enfoque facilita la creación de jerarquías de objetos y es el más utilizado en proyectos actuales por su claridad y compatibilidad con otros lenguajes orientados a objetos.

Ejemplo de ejecución:



```
> // Autor: Mario Segura Abad
// Fecha: 08/10/2025

// -----
// SINTAXIS CLASS
// -----
class Enemigo {
  constructor(energia = 10) {
    this.energia = energia;
  }

  mover() {
    console.log(`Propiedad de Enemigo: ${this.energia}`);
  }
}

// Clase derivada (Orco) que hereda de Enemigo
class Orco extends Enemigo {
  constructor(energia = 10, fuerza = 15) {
    super(energia); // Llama al constructor de Enemigo
    this.fuerza = fuerza;
  }

  disparar() {
    console.log(`Propiedad de Orco: ${this.fuerza}`);
  }
}

// Probar métodos
enemigo.mover(); // Propiedad de Enemigo: 10
orco.mover(); // Propiedad de Enemigo: 10
orco.disparar(); // Propiedad de Orco: 15
troll.mover(); // Propiedad de Enemigo: 10
troll.golpear(); // Propiedad de Troll: 20

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 08/10/2025");
Propiedad de Enemigo: 10
Propiedad de Enemigo: 10
Propiedad de Orco: 15
Propiedad de Enemigo: 10
Propiedad de Troll: 20
Autor: Mario Segura Abad
Fecha de realización: 08/10/2025
```

2. Funciones constructoras

Antes de la introducción de `class`, JavaScript utilizaba funciones constructoras para crear objetos. En este modelo, se define una función que asigna propiedades al objeto mediante `this`, y se añaden métodos al prototipo para que puedan ser compartidos entre instancias.

3. `Object.create()`

Este método permite crear un nuevo objeto enlazado directamente a otro como prototipo. Es el enfoque más flexible y directo para establecer herencia sin necesidad de constructores ni clases.

- **Ponte a prueba – `this`**

Este ejercicio se ha centrado en el análisis del comportamiento de `this`, una de las características más complejas y dinámicas de JavaScript. A diferencia de otros lenguajes, `this` en JavaScript no se refiere al objeto donde se define la función, sino al objeto desde el que se invoca.

Ejemplo de ejecución:

```

top ▼ Filter

> // Autor: Mario Segura Abad
// Fecha: 09/10/2025

// -----
// EJERCICIO: Ponte a Prueba 3
// -----

function Persona(nombre, apellido) {
  this.nombre = nombre;
  this.apellido = apellido;

  // Método saludar: muestra el nombre y apellido del objeto que llame al método
  this.saludar = function () {
    return `${this.nombre} ${this.apellido}`;
  }
}

// Creamos tres objetos Persona
let persona1 = new Persona("Ana", "Pérez");
let persona2 = new Persona("Lucía", "Martínez");
let persona3 = new Persona("Inés", "González");

// -----
// bind(): crea una nueva función donde "this"
// queda vinculado permanentemente al objeto indicado.
// En este caso, "this" apuntará SIEMPRE a persona3
// -----
let saludo1 = persona1.saludar.bind(persona3);

// 3. Usamos .call(persona3): forzamos que "this" sea persona3
// Resultado: "Inés González"
persona1.saludar.call(persona3);

// 4. Usamos .apply(persona2): igual que call, pero con argumentos en arraya
// Resultado: "Lucía Martínez"
persona1.saludar.apply(persona2);

// 5. saludo1() fue creado con bind(persona3), así que "this" siempre será persona3
// Resultado: "Inés González"
saludo1();

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 09/10/2025");

// 6. saludo2() fue copiado sin bind, por lo que al ejecutarse sin objeto pierde el "this"
// En modo estricto -> da error porque this = undefined
// En modo no estricto (por defecto en navegadores) -> this = window
// window no tiene propiedades "nombre" ni "apellido"
// Resultado: "undefined undefined"
saludo2();

Autor: Mario Segura Abad
Fecha de realización: 09/10/2025
< 'undefined undefined'

```

Cómo visualizarlo

Requisitos:

- Editor de código: en mi caso, Visual Studio Code.
- Navegador web para scripts con `document.write()` o `prompt()`.
- Node.js o la consola del navegador (F12) para ejecutar scripts con `console.log()`.

Pasos:

1. Abrir los archivos `.js` o `.html` en el editor de código.
2. Ejecutar los scripts en el navegador o consola según el tipo.
3. Observar el resultado en pantalla o consola.
4. Revisar las capturas de ejecución incluidas en la carpeta correspondiente.

Objetivo de la práctica

- Probar el código del material teórico del Apartado 3.
- Ejecutar y analizar los scripts proporcionados en la tarea.
- Crear objetos usando clases (en el caso de mi práctica), funciones constructoras y `Object.create()`.
- Analizar el comportamiento de `this` en distintos contextos.
- Documentar correctamente cada ejercicio con nombre, fecha y capturas.
- Aplicar buenas prácticas de programación y presentación técnica.
- Entregar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido entender cómo se crean objetos en JavaScript y cómo se estructuran los prototipos.

Se ha comprobado que existen varias formas de implementar herencia y reutilización de métodos. Además, se ha analizado el comportamiento dinámico de `this`, reforzando la importancia del contexto en la ejecución de funciones.

La práctica, asimismo, útil para consolidar conceptos clave de la Programación Orientada a Objetos en JavaScript.